

Statistical Functions

NumPy has quite a few useful statistical functions for finding minimum, maximum, percentile, standard deviation and variance, etc. from the given elements in the array.

The functions are explained as follows :

1. `numpy.amin()` and `numpy.amax()`

These functions return the minimum and the maximum from the elements in the given array along the specified axis.

```
import numpy as np
a = np.array([[3,7,5],[8,4,3],[2,4,9]])
print('Our array is:')
print(a)
print('\n')
print('Applying amin() function:')
print(np.amin(a, 1))
print('\n')

print('Applying amin() function again:')
print(np.amin(a, 0))
print('\n')

print('Applying amax() function:')
```

```
print(np.amax(a))
```

```
print('\n')
```

```
print('Applying amax() function again:')
```

```
print(np.amax(a, axis=0))
```

output:

Our array is:

```
[[3 7 5]
```

```
 [8 4 3]
```

```
 [2 4 9]]
```

Applying amin() function:

```
[3 3 2]
```

Applying amin() function again:

```
[2 4 3]
```

Applying amax() function:

```
9
```

Applying amax() function again:

```
[8 7 9]
```

2. numpy.ptp()

The numpy.ptp() function returns the range (maximum-minimum) of values along an axis. numpy.ptp() function plays an important role in statistics by finding out Range of given numbers.

Range = max value – min value.

```
import numpy as np
a = np.array([[3,7,5],[8,4,3],[2,4,9]])
print('Our array is:')
print(a)
print('\n')

print('Applying ptp() function:')
print(np.ptp(a))
print('\n')

print('Applying ptp() function along axis 1:')
print(np.ptp(a, axis=1))
print('\n')

print('Applying ptp() function along axis 0:')
print(np.ptp(a, axis=0))
```

output:

Our array is:

```
[[3 7 5]
 [8 4 3]
 [2 4 9]]
```

Applying ptp() function:

7

Applying ptp() function along axis 1:

```
[4 5 7]
```

Applying ptp() function along axis 0:

```
[6 3 6]
```

3. numpy.mean()

Arithmetic mean is the sum of elements along an axis divided by the number of elements. The numpy.mean() function returns the arithmetic mean of elements in the array. If the axis is mentioned, it is calculated along it.

```
import numpy as np
a = np.array([[1,2,3],[3,4,5],[4,5,6]])

print('Our array is:')
print(a)
print('\n')

print('Applying mean() function:')
print(np.mean(a))
print('\n')

print('Applying mean() function along axis 0:')
print(np.mean(a, axis=0))
print('\n')
```



```
print('Applying mean() function along axis 1:')  
print(np.mean(a, axis=1))
```

output:

Our array is:

```
[[1 2 3]  
 [3 4 5]  
 [4 5 6]]
```

Applying mean() function:

```
3.6666666666666665
```

Applying mean() function along axis 0:

```
[2.66666667 3.66666667 4.66666667]
```

Applying mean() function along axis 1:

```
[2. 4. 5.]
```

4. `numpy.median()`

Median is defined as the value separating the higher half of a data sample from the lower half.

```
import numpy as np  
a = np.array([[30, 65, 70], [80, 95, 10], [50, 90, 60]])  
print('Our array is:')  
print(a)
```

```
print('\n')
```

```
print('Applying median() function:')
```

```
print(np.median(a))
```

```
print('\n')
```

```
print('Applying median() function along axis 0:')
```

```
print(np.median(a, axis=0))
```

```
print('\n')
```

```
print('Applying median() function along axis 1:')
```

```
print(np.median(a, axis=1))
```

output:

Our array is:

```
[[30 65 70]
```

```
 [80 95 10]
```

```
 [50 90 60]]
```

Applying median() function:

65.0

Applying median() function along axis 0:

```
[50. 90. 60.]
```

Applying median() function along axis 1:

[65. 80. 60.]

5. Standard Deviation

Standard deviation is the square root of the average of squared deviations from mean.

```
import numpy as np
print(np.std([1, 2, 3, 4]))
```

output:

1.118033988749895

6. Variance

Variance is the average of squared deviations, i.e., $\text{mean}(\text{abs}(x - x.\text{mean}())^2)$. In other words, the standard deviation is the square root of variance.

```
import numpy as np
print(np.var([1, 2, 3, 4]))
```

output:

1.25

Sort, Search & Counting Functions

A variety of sorting related functions are available in NumPy. These sorting functions implement different sorting algorithms, each of them characterized

by the speed of execution, worst case performance, the workspace required and the stability of algorithms.

- 'quicksort'
- 'mergesort'
- 'heapsort'

1. numpy.sort()

The sort() function returns a sorted copy of the input array. It has the following parameters –

Syntax:

```
numpy.sort(a, axis, kind, order)
```

- **a** : Array to be sorted
- **axis** : The axis along which the array is to be sorted. If none, the array is flattened, sorting on the last axis
- **kind** : Default is quicksort
- **order** : If the array contains fields, the order of fields to be sorted

```
import numpy as np
a = np.array([[3, 7], [9, 1]])
print('Our array is:')
print(a)
print('Applying sort() function:')
```

```
print(np.sort(a))
```

```
print('Sort along axis 0:')
```

```
print(np.sort(a, axis=0))
```

```
# Order parameter in sort function
```

```
dt = np.dtype([('name', 'S10'), ('age', int)])
```

```
a = np.array([("raju", 21), ("anil", 25), ("ravi", 17),  
("amar", 27)], dtype=dt)
```

```
print('Our array is:')
```

```
print(a)
```

```
print('Order by name:')
```

```
print(np.sort(a, order='name'))
```

output:

Our array is:

```
[[3 7]  
 [9 1]]
```

Applying sort() function:

```
[[3 7]  
 [1 9]]
```

Sort along axis 0:

```
[[3 1]  
 [9 7]]
```

Our array is:



```
[(b'raju', 21) (b'anil', 25) (b'ravi', 17) (b'amar', 27)]
```

Order by name:

```
[(b'amar', 27) (b'anil', 25) (b'raju', 21) (b'ravi', 17)]
```

2. `numpy.argmax()` and `numpy.argmin()`

These two functions return the indices of maximum and minimum elements respectively along the given axis.

`np.argmax()` GETS THE INDEX
OF THE MAXIMUM VALUE OF A NUMPY ARRAY

	1	2	3	100	5
Index:	0	1	2	3	4

```
import numpy as np
a = np.array([[30, 40, 70], [80, 20, 10], [50, 90, 60]])
print('Our array is:')
print(a)
print('\n')
print('Applying argmax() function:')
print(np.argmax(a))
print('\n')
```

```
print('Index of maximum number in flattened array')
print(a.flatten())
print('\n')

print('Array containing indices of maximum along axis 0:')
maxindex = np.argmax(a, axis=0)
print(maxindex)
print('\n')

print('Array containing indices of maximum along axis 1:')
maxindex = np.argmax(a, axis=1)
print(maxindex)
print('\n')

print('Applying argmin() function:')
minindex = np.argmin(a)
print(minindex)
print('\n')

print('Flattened array:')
print(a.flatten()[minindex])
print('\n')

print('Flattened array along axis 0:')
minindex = np.argmin(a, axis=0)
print(minindex)
```



```
print('\n')
```

```
print('Flattened array along axis 1:')
```

```
minindex = np.argmin(a, axis=1)
```

```
print(minindex)
```

output:

Our array is:

```
[[30 40 70]
```

```
 [80 20 10]
```

```
 [50 90 60]]
```

Applying argmax() function:

7

Index of maximum number in flattened array

```
[30 40 70 80 20 10 50 90 60]
```

Array containing indices of maximum along axis 0:

```
[1 2 0]
```

Array containing indices of maximum along axis 1:

```
[2 0 1]
```

Applying argmin() function:

5

Flattened array:

10

Flattened array along axis 0:

[0 1 1]

Flattened array along axis 1:

[0 2 0]

3. `numpy.nonzero()`

The `numpy.nonzero()` function returns the indices of non-zero elements in the input array.

```
import numpy as np
a = np.array([[30,40,0],[0,20,10],[50,0,60]])
print('Our array is:')
print(a)
print('\n')
print('Applying nonzero() function:')
print(np.nonzero(a))
```

output:

Our array is:

[[30 40 0]

[0 20 10]

[50 0 60]]

Applying nonzero() function:

```
(array([0, 0, 1, 1, 2, 2], dtype=int64), array([0, 1, 1, 2, 0, 2], dtype=int64))
```

4. numpy.where()

The where() function returns the indices of elements in an input array where the given condition is satisfied.

```
import numpy as np
x = np.arange(9.).reshape(3, 3)
print('Our array is:')
print(x)

print('Indices of elements > 3')
y = np.where(x > 3)
print(y)

print('Use these indices to get elements satisfying the condition')
print(x[y])
```

output:

Our array is:

```
[[0. 1. 2.]
 [3. 4. 5.]
```

```
[6. 7. 8.]
```

Indices of elements > 3

```
(array([1, 1, 2, 2, 2], dtype=int64), array([1, 2, 0, 1, 2],  
dtype=int64))
```

Use these indices to get elements satisfying the condition

```
[4. 5. 6. 7. 8.]
```

5. numpy.extract()

The extract() function returns the elements satisfying any condition.

```
import numpy as np  
x = np.arange(9.).reshape(3, 3)  
print('Our array is:')  
print(x)  
  
# define a condition  
condition = np.mod(x,2) == 0  
  
print('Element-wise value of condition')  
print(condition)  
  
print('Extract elements using condition')  
print(np.extract(condition, x))
```

output:

Our array is:

```
[[0. 1. 2.]  
 [3. 4. 5.]  
 [6. 7. 8.]]
```

Element-wise value of condition

```
[[ True False  True]  
 [False  True False]  
 [ True False  True]]
```

Extract elements using condition

```
[0. 2. 4. 6. 8.]
```

Matrix Library

NumPy package contains a Matrix library `numpy.matlib`. This module has functions that return matrices instead of `ndarray` objects.

1. `numpy.matlib.zeros()`

This function returns the matrix filled with zeros .

```
import numpy.matlib #import the optional subpackage which is  
not accessible directly otherwise error will be thrown  
import numpy as np  
print(np.matlib.zeros((2,2)))
```

output:

```
[[0. 0.]  
 [0. 0.]]
```

2. numpy.matlib.ones()

This function returns the matrix filled with 1s.

```
import numpy.matlib
import numpy as np
print (np.matlib.ones((2,2)))
```

output:

```
[[1.  1.]
 [1.  1.]]
```

3. numpy.matlib.identity()

The numpy.matlib.identity() function returns the Identity matrix of the given size. An identity matrix is a square matrix with all diagonal elements as 1.

```
import numpy.matlib
import numpy as np
print (np.matlib.identity(5, dtype = float))
```

output:

```
[[1.  0.  0.  0.  0.]
 [0.  1.  0.  0.  0.]
 [0.  0.  1.  0.  0.]
```

```
[0. 0. 0. 1. 0.]  
[0. 0. 0. 0. 1.]]
```

4. `numpy.matlib.rand()`

The `numpy.matlib.rand()` function returns a matrix of the given size filled with random values.

```
import numpy.matlib  
import numpy as np  
print(np.matlib.rand(3,3))
```

output:

```
[[0.07615247 0.9230499 0.77497548]  
 [0.37525869 0.325195 0.28600785]  
 [0.34291161 0.02614019 0.9702495 ]]
```

Linear Algebra

NumPy package contains `numpy.linalg` module that provides all the functionality required for linear algebra.

1. `numpy.dot()`

This function returns the dot product of two arrays.

For 2-D vectors, it is the equivalent to matrix multiplication. For 1-D arrays, it is the inner product of the vectors.


```
import numpy as np
a = np.array([[1,2],[3,4]])
b = np.array([[11,12],[13,14]])
print("Array a")
print(a)
print("Array b")
print(b)
print("Dot product")
print(np.dot(a,b))
```

Note that the dot product is calculated as -
*#[[1*11+2*13, 1*12+2*14],[3*11+4*13, 3*12+4*14]]*

output:

Array a

*[[1 2]
 [3 4]]*

Array b

*[[11 12]
 [13 14]]*

Dot product

*[[37 40]
 [85 92]]*



smectechologies
GUIDING THE FUTURE WITH AI
II Floor, Express House, Kaloor, Kochi, Ernakulam, Kerala 682017
Ph. No: +91 9778191215

2. numpy.vdot() : This function returns the dot product of the two vectors. If the first argument is complex, then its conjugate is used for calculation. If the argument id is multi-dimensional array, it is flattened.

```
import numpy as np
a = np.array([[1,2],[3,4]])
b = np.array([[11,12],[13,14]])
print("Array a:")
print(a)
print("Array b:")
print(b)
print("V dot product:")
print(np.vdot(a,b))
# Note -  $1*11 + 2*12 + 3*13 + 4*14 = 130$ 
```

output:

Array a:

```
[[1 2]
 [3 4]]
```

Array b:

```
[[11 12]
 [13 14]]
```

V dot product:

130

3. numpy.inner()

This function returns the inner product of vectors for 1-D arrays.

For higher dimensions, it returns the sum product over the last axes.

```
import numpy as np
print (np.inner(np.array([1,2,3]),np.array([0,1,0])) )
# Equates to 1*0+2*1+3*0
```

output:

2

4. numpy.matmul()

The numpy.matmul() function returns the matrix product of two arrays. While it returns a normal product for 2-D arrays.

For 2-D array, it is matrix multiplication

```
import numpy as np
a = [[1,0],[0,1]]
b = [[4,1],[2,2]]
print("Matrix a")
print(a)
print("Matrix b")
print(b)
print("Result:")
print (np.matmul(a,b))
```

output:

Matrix a

```
[[1, 0], [0, 1]]
```

Matrix b

```
[[4, 1], [2, 2]]
```

Result:

```
[[4 1]
```

```
 [2 2]]
```

5. numpy.linalg.det()

Determinant is a very useful value in linear algebra. It is calculated from the diagonal elements of a square matrix.

For a 2x2 matrix, it is simply the subtraction of the product of the top left and bottom right element from the product of the other two. In other words, for a matrix $\begin{bmatrix} a & b \\ c & d \end{bmatrix}$, the determinant is computed as 'ad-bc'. The larger square matrices are considered to be a combination of 2x2 matrices.

`numpy.linalg.det()` function calculates the determinant of the input matrix.

```
import numpy as np
a = np.array([[1,2], [3,4]])
print (np.linalg.det(a))
```

output:

```
-2.0000000000000004
```

6. numpy.linalg.solve()

The `numpy.linalg.solve()` function gives the solution of linear equations in the matrix form.

Considering the following linear equations –

$$3x + 4y + 5z = 9$$

$$y + 2x + 3z = 8$$

$$2x + 4y + 5z = 7$$

Program to show the working of solve()

```
import numpy as np
```

creating the array "a"

```
A = np.array([[3, 4, 5], [1, 2, 3], [2, 4, 5]])
```

```
B = np.array([9, 8, 7])
```

```
print("Array A is: \n", A)
```

```
print("Array B is : \n", B)
```

Calculating the equation

```
ans = np.linalg.solve(A, B)
```

Printing the answer

```
print("Answer of the equation is : \n", ans)
```

Checking if the answer is correct

```
print(np.allclose(np.dot(A, ans), B))
```

output:

Array A is:

```
[[3 4 5]
```

```
[1 2 3]
```

```
[2 4 5]]
```

Array B is :

```
[9 8 7]
```

Answer of the equation is :

```
[ 2. -10.5  9. ]
```

True

7. numpy.linalg.inv()

We use `numpy.linalg.inv()` function to calculate the inverse of a matrix. The inverse of a matrix is such that if it is multiplied by the original matrix, it results in identity matrix.

```
import numpy as np
```

```
x = np.array([[1,2],[3,4]])
```

```
y = np.linalg.inv(x)
```

```
print(x)
```

```
print(y)
```

```
print(np.dot(x, y))
```

output:

```
[[1 2]
```

```
 [3 4]]
```

```
[[ -2.   1. ]
```

```
 [ 1.5 -0.5]]
```

```
[[1.0000000e+00  0.0000000e+00]
```

```
 [8.8817842e-16  1.0000000e+00]]
```